

SMART HAS

Evolução Técnica e Conectada

AI Logistics Extension — React Native · Spring Boot · Angular

Documentação da Atividade

FIAP — [Nome da Disciplina/Fase]

Integrantes do grupo

Nome completo	RM	Turma
[Preencher nome do integrante 1]	[RM]	[Turma]
[Preencher nome do integrante 2]	[RM]	[Turma]
[Preencher nome do integrante 3]	[RM]	[Turma]
[Preencher nome do integrante 4]	[RM]	[Turma]

Observação: adicione ou remova linhas conforme o número real de integrantes do grupo.

1. Contextualização

O Smart HAS é um sistema inteligente de apoio à decisão voltado ao acompanhamento de pacientes com Hipertensão Arterial Sistêmica (HAS), alinhado aos princípios da Sociedade 5.0 — tecnologia a serviço do cuidado centrado na pessoa. Nesta fase, o projeto evolui com a camada AI Logistics Extension: um motor de priorização que sugere, de forma automática, a urgência e a janela ideal de entrega de medicamentos a partir do risco clínico calculado com base nas leituras de pressão arterial mais recentes de cada paciente.

O repositório do projeto (LeoMarsiarelli/Smart-HAS) iniciou esta fase sem código de fases anteriores versionado. Diante disso, o grupo optou por tratar esta etapa como a consolidação da base técnica completa do sistema — mobile, back-end e dashboard administrativo — já incorporando as práticas e integrações exigidas pelos capítulos desta fase, descritas a seguir.

2. Parte 1 — Escolha da Stack Tecnológica e Justificativa

2.1 Decisão: Opção A — React Native

O grupo optou por construir o cliente mobile em React Native (com Expo), migrando/consolidando as funcionalidades do Smart HAS nessa stack.

Justificativa técnica

- Padronização de paradigma entre front-ends: React Native (mobile) e Angular (dashboard web) compartilham conceitos de componentização, data binding reativo e consumo de API REST, reduzindo a curva de aprendizado do time e o custo de manutenção de dois times de front-end separados.
- Expo acelera o ciclo de desenvolvimento e a demonstração da atividade: o app roda em dispositivo físico via Expo Go, sem exigir Android Studio/Xcode configurados por todos os integrantes do grupo.
- Ecossistema maduro (React Navigation, bibliotecas de UI e integração HTTP) é um investimento direto no roadmap da AI Logistics Extension — por exemplo,

notificações push de status de entrega e mapas de rota em fases futuras.

- Componentização explícita com os elementos nativos View, Text, Image e Button, e navegação estruturada com React Navigation (pilha de autenticação + abas principais), conforme exigido pela atividade.

Alinhamento com os objetivos do projeto

A escolha reforça o objetivo central do Smart HAS — apoiar decisões de saúde e logística de forma acessível — ao entregar um app leve, multiplataforma (Android/iOS) e de fácil distribuição para demonstração, ao mesmo tempo em que mantém consistência arquitetural com o dashboard administrativo em Angular, ambos consumindo o mesmo contrato de API REST do backend Spring Boot.

Roadmap tecnológico

Status	Entrega
Concluído em fases anteriores	Definição do escopo estratégico do Smart HAS, planejamento da arquitetura geral e protótipos iniciais de app mobile.
Concluído nesta fase	App mobile completo em React Native (login, dashboard, leituras de pressão, AI Logistics, perfil); API REST completa em Spring Boot com autenticação JWT, persistência e motor de priorização; dashboard administrativo em Angular integrado à mesma API.
Planejado para próximas fases	Evolução do motor de regras para um modelo de Machine Learning treinado com dados históricos reais; notificações push; deploy em nuvem (contêiner + banco gerenciado); testes E2E; papéis de acesso adicionais (ex.: cuidador/familiar).

3. Parte 2 — Back-end Escalável com Spring Boot

O back-end foi desenvolvido em Java 17 com Spring Boot 3, seguindo a separação em camadas Controller → Service → Repository → Model, com DTOs dedicados para nunca expor entidades JPA diretamente pela API.

3.1 API REST

Recurso	Endpoints
Autenticação	POST /api/auth/register · POST /api/auth/login
Usuário	GET /api/users/me
Leituras de pressão	GET/POST /api/readings · GET/PUT/DELETE /api/readings/{id}
AI Logistics (entregas)	GET/POST /api/deliveries · GET/DELETE /api/deliveries/{id} · PATCH /api/deliveries/{id}/status

3.2 Autenticação e autorização

Autenticação stateless via JWT (JSON Web Token), com filtro dedicado (JwtAuthenticationFilter) que valida o token em cada requisição e popula o contexto de segurança do Spring. Senhas são armazenadas com hash BCrypt. Papéis (PATIENT/ADMIN) controlam o escopo de acesso: pacientes só acessam os próprios dados; administradores acessam a base completa.

3.3 Persistência e integração

Banco de dados relacional via Spring Data JPA: H2 em arquivo local para desenvolvimento/demonstração, com perfil alternativo (application-postgres.yml) pronto para PostgreSQL em produção. Todas as operações CRUD de leituras e pedidos de entrega estão implementadas, com validação via Bean Validation e tratamento de erros centralizado (GlobalExceptionHandler), retornando um formato padronizado de erro para toda a API.

3.4 AI Logistics Extension — motor de priorização

O serviço LogisticsAiService implementa um motor de regras (IA simbólica / expert system) que calcula, no momento da criação de um pedido de entrega, o risco do paciente, a prioridade logística e a janela estimada de entrega — com base na classificação da leitura de pressão arterial mais recente. É uma escolha deliberada por uma IA 100% explicável e auditável nesta fase, com roadmap para evoluir para um modelo de Machine Learning treinado com dados históricos reais.

Classificação da última leitura	Risco	Prioridade	Janela sugerida
Crise hipertensiva	95	URGENTE	+1h a +2h
HAS estágio 2	75	ALTA	+2h a +4h
HAS estágio 1	50	MÉDIA	+4h a +8h
Elevada	30	BAIXA	Próximo dia útil
Normal / sem leitura	15	BAIXA	Próximo dia útil

3.5 Documentação e testes

A API é documentada interativamente via Swagger UI (springdoc-openapi), disponível em [/swagger-ui.html](#), com especificação OpenAPI em [/v3/api-docs](#). Testes automatizados cobrem a inicialização do contexto Spring e as regras de classificação de pressão arterial (mvn test).

4. Parte 3 — Integração Web com Angular

O dashboard administrativo foi implementado em Angular 17 (standalone components), consumindo exatamente a mesma API REST utilizada pelo app mobile.

4.1 Estrutura e navegação

- Rotas configuradas: `/login`, `/home` (visão geral) e `/admin` (gestão de leituras e entregas), com redirecionamento padrão e rota curinga.
- Serviços Angular dedicados (`AuthService`, `ReadingService`, `DeliveryService`) usando `HttpClient`, com um interceptor HTTP que anexa automaticamente o token JWT (`Authorization: Bearer`) a cada requisição.
- Guards de rota protegendo `/home` (usuário autenticado) e `/admin` (apenas ADMIN).

4.2 Data binding e diretivas

O dashboard utiliza as quatro formas de data binding do Angular:

- Interpolação `{{ }}` para exibir dados (ex.: contagens por classificação/prioridade na tela inicial).
- Property binding `[ ]` (ex.: `[class.active]`, `[disabled]` nos formulários e filtros).
- Event binding `( )` (ex.: `(click)` para logout, exclusão e atualização de status).
- Two-way binding `[( )]` com `ngModel` no formulário de criação de pedido de entrega e nos filtros da tela administrativa.

As diretivas estruturais `*ngIf` (estados de carregamento, erro, vazio e controle de acesso por papel) e `*ngFor` (listagem de leituras e pedidos de entrega em tabelas) são usadas extensivamente nas telas `/home` e `/admin`.

4.3 Formulário funcional com ngModel

A tela `/admin` inclui um formulário completo, orientado por template (template-driven form) com `[(ngModel)]`, para criar um novo pedido de entrega de medicamento — reaproveitando o mesmo endpoint `POST /api/deliveries` do app mobile — além de filtros por classificação/prioridade/status também construídos com `ngModel`.

4.4 Estilização e feedback

A interface utiliza CSS próprio (sem dependência de frameworks de UI), com barra de navegação sensível ao estado de login, indicadores de carregamento, banners de erro e confirmações de sucesso após cada ação (criação, atualização de status e exclusão).

5. Entrega do Projeto

5.1 Código-fonte

Repositório GitHub: <https://github.com/LeoMarsiarelli/Smart-HAS>

Estrutura do monorepo: /mobile (React Native), /backend (Spring Boot), /web-admin (Angular) e /docs (contrato de API e esta documentação).

5.2 Vídeo de demonstração

Link do vídeo (YouTube, não listado): [INSERIR LINK DO VÍDEO AQUI]

Lembrete: publicar o vídeo como "não listado" no YouTube e garantir que o link esteja acessível para a correção do professor, incluindo demonstração prática do app em funcionamento.

5.3 Credenciais de demonstração

Papel	Email	Senha
Administrador	admin@smarthas.com	admin123
Paciente	maria@smarthas.com	paciente123

6. Conclusão

Nesta fase, o Smart HAS evoluiu de um escopo planejado para uma base técnica completa e integrada: um app mobile em React Native, uma API REST robusta e segura em Spring Boot com uma camada de IA de logística explicável, e um dashboard administrativo em Angular consumindo o mesmo contrato de dados. O resultado é uma arquitetura coesa, pronta para evoluir — nas próximas fases — para um motor de priorização baseado em Machine Learning e para operação em ambiente de nuvem, mantendo o foco em apoiar decisões de saúde de forma acessível e alinhada à Sociedade 5.0.